# ON THE ROLE OF PLANNING IN MODEL-BASED DEEP REINFORCEMENT LEARNING

Jessica B Hamrick, Abram L. Friesen, Feryal Behbahani, Arthur Guez, Fabio Viola, Sims Witherspoon, Thomas Anthony, Lars Holger Buesing, Petar Velickovic, Theophane Weber

Name: Hongming Zhang
Date: 2022.12.09

Hamrick, Jessica B., et al. "On the role of planning in model-based deep reinforcement learning." International Conference on Learning Representations. 2021.

# Introduction

- ICLR 2021

The diversity of model-based methods make it difficult to track which components drive success and why. This paper focuses on three questions:

- How does planning benefit model-based reinforcement learning (MBRL) agents?
- Within planning, what choices drive performance?
- To what extent does planning improve generalization?

Based on ablations of MuZero across a wide range of environments, results suggest:

- Planning is most useful in the learning process, both for policy updates and for providing a more useful data distribution.
- Using shallow trees with simple Monte-Carlo rollouts is as performant as more complex methods, except in the most difficult reasoning tasks.
- Planning alone is insufficient to drive strong generalization.

# Model-Based RL

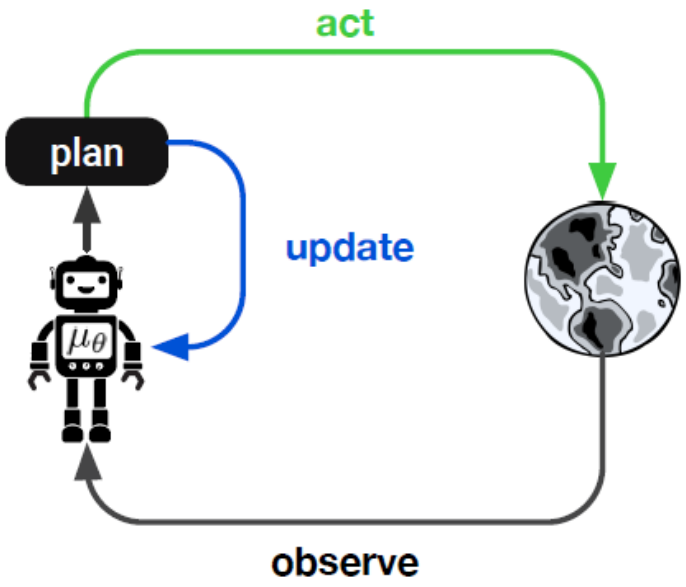



Figure 1: Model-based approximate policy iteration. The agent **updates** its policy using targets computed via planning and optionally **acts** via planning during training, at test time, or both.

- Learning: deep learning of a model, policy, and/or value function.
- Planning: using a learned or given model to construct trajectories or plans.

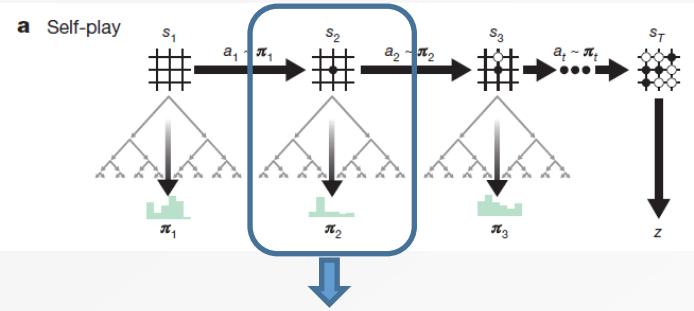
- Decision-time planning: use the model to select actions (MPC)
- Background planning: use the model to update a policy (Dyna)

- Better learning usually resulting in better planning, and better planning resulting in better learning.

# MCTS in AlphaGo Zero

a) Select
b) Expand and evaluate
c) Backup

d) Play




a)

$$a = \text{argmax}_a(Q(s,a) + U(s,a))$$

Exploration : $U(s,a) = c_{puct} P(s,a) \frac{\sqrt{\sum_b N(s,b)}}{1+N(s,a)}$,

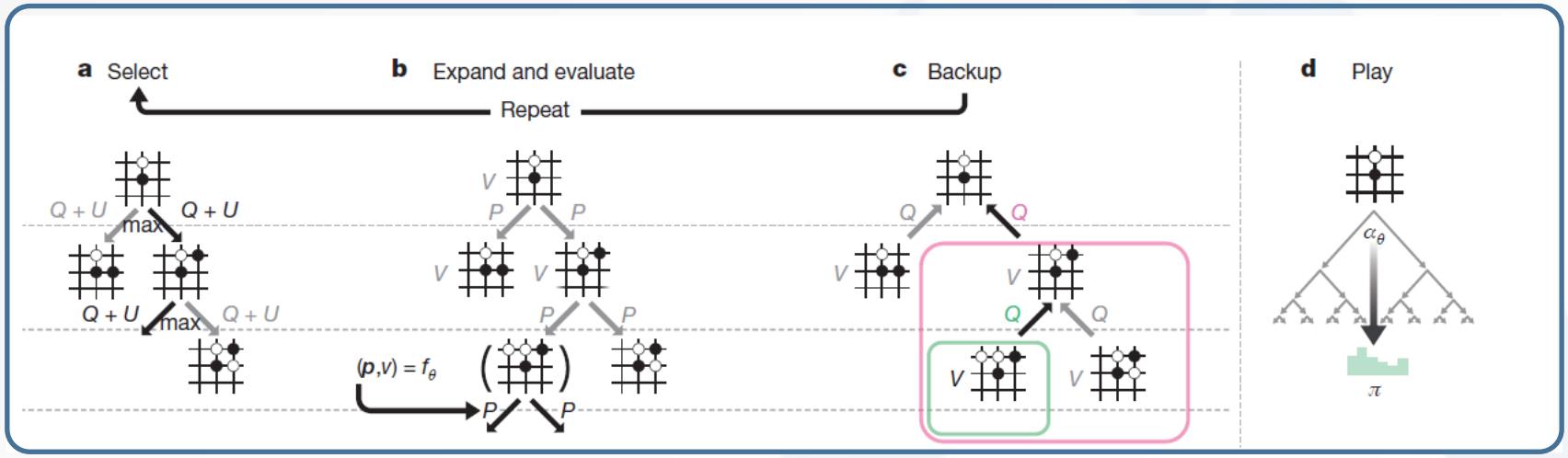
Exploitation : $Q(s,a) = \frac{W}{N}$

c)

$$N(s,a) = N(s,a) + 1$$
$$W(s,a) = W(s,a) + v(s')$$
$$Q(s,a) = \frac{W(s,a)}{N(s,a)}$$

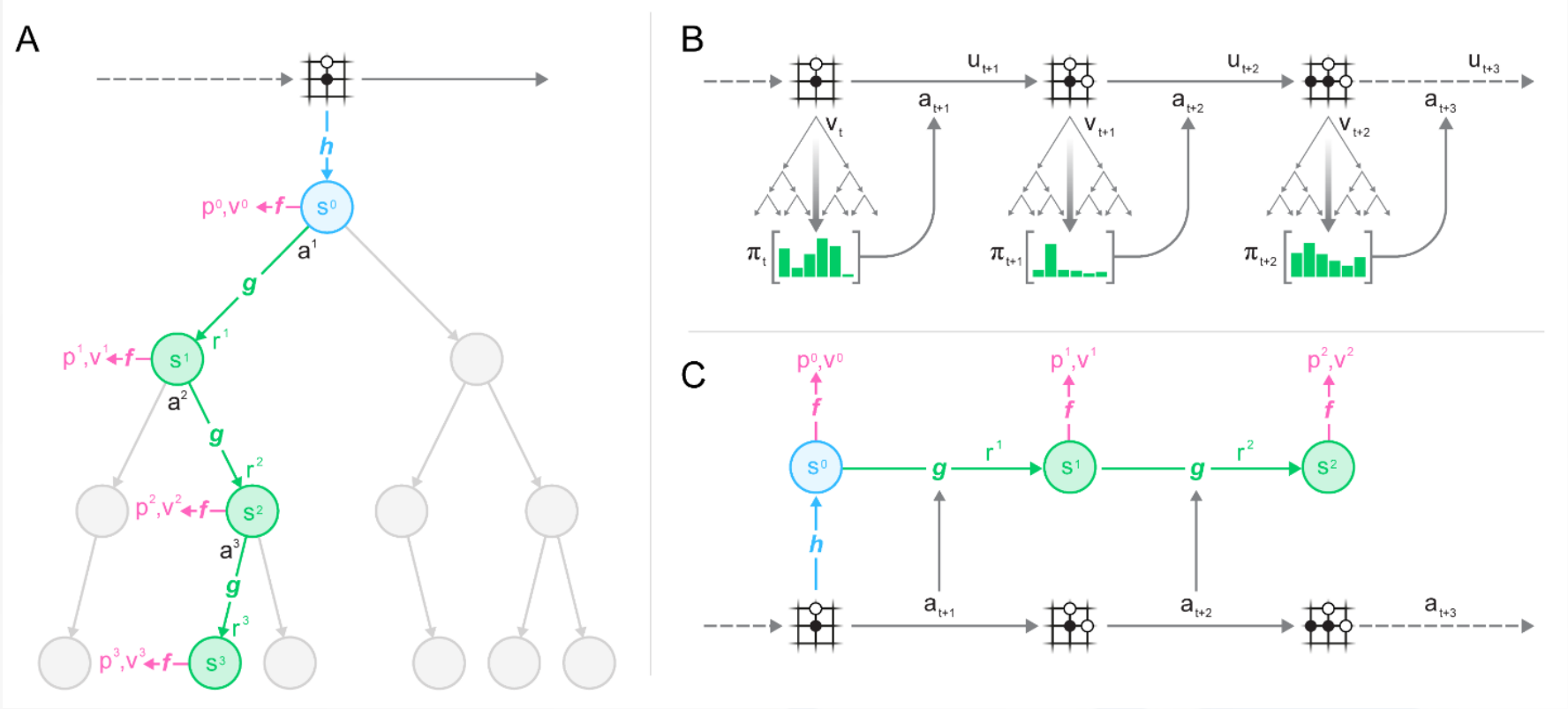
d)

$\pi(a|s) = \frac{N(s,a)^{1/\tau}}{\sum_b N(s,b)^{1/\tau}}$, $\tau$ is a temperature parameter



# MuZero

(A) How MuZero uses its model to plan. (B) How MuZero acts in the environment. (C) How MuZero trains its model.




- Model

  Encoder: $s_t^0 = h_\theta(o_1, \cdots, o_t)$

  Dynamics: $r_{\theta,t}^k, s_t^k = g_\theta(s_t^{k-1}, a_t^{k-1})$

  Prior: $\pi_{\theta,t}^k, v_{\theta,t}^k = f_\theta(s_t^k)$

- MCTS: PUCT (AlphaZero style)

$$a = \text{argmax}_a(Q(s,a) + U(s,a))$$

$$U(s,a) = c_{puct} \cdot \pi(s,a) \sqrt{\frac{\sum_b N(s,b)}{1+N(s,a)}}$$

$$l_t(\theta) = \sum_{k=0}^{K} l^r\left(r_{\theta,t}^k, r_{t+k}^{\text{env}}\right) + l^v\left(v_{\theta,t}^k, z_{t+k}\right) + l^p\left(\pi_{\theta,t}^k, \pi_{t+k}^{\text{MCTS}}\right) + c\|\theta\|^2$$

# Experimental Setting

$D_{tree}$: the maximum depth we search within the tree

$D_{UCT}$: the maximum depth we optimize the exploration-exploitation tradeoff via $p_{UCT}$

B: the search budget

Model: learned (default) or environment simulator

Planning algorithm: MCTS or breadth-first search

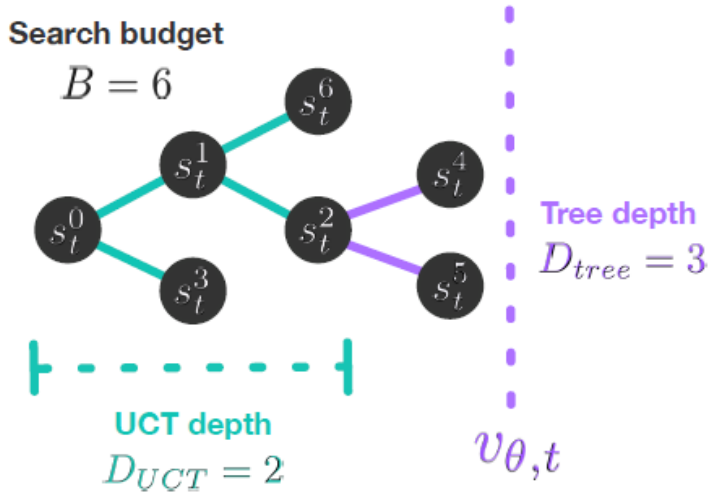



Figure 2: For nodes at depth $d < D_{\mathrm{UCT}}$, we select actions according to pUCT (Section 2), while for nodes at depth $D_{\mathrm{UCT}} \leq d < D_{\mathrm{tree}}$, we select actions by sampling from $\pi_{\theta,t}$. Nodes at depth $d = D_{\mathrm{tree}}$ (and deeper) are not expanded; instead, we stop the search and backup using $v_{\theta,t}$. The search budget $B$ is equal to the number of nodes in the tree aside from the root $s_t^0$.

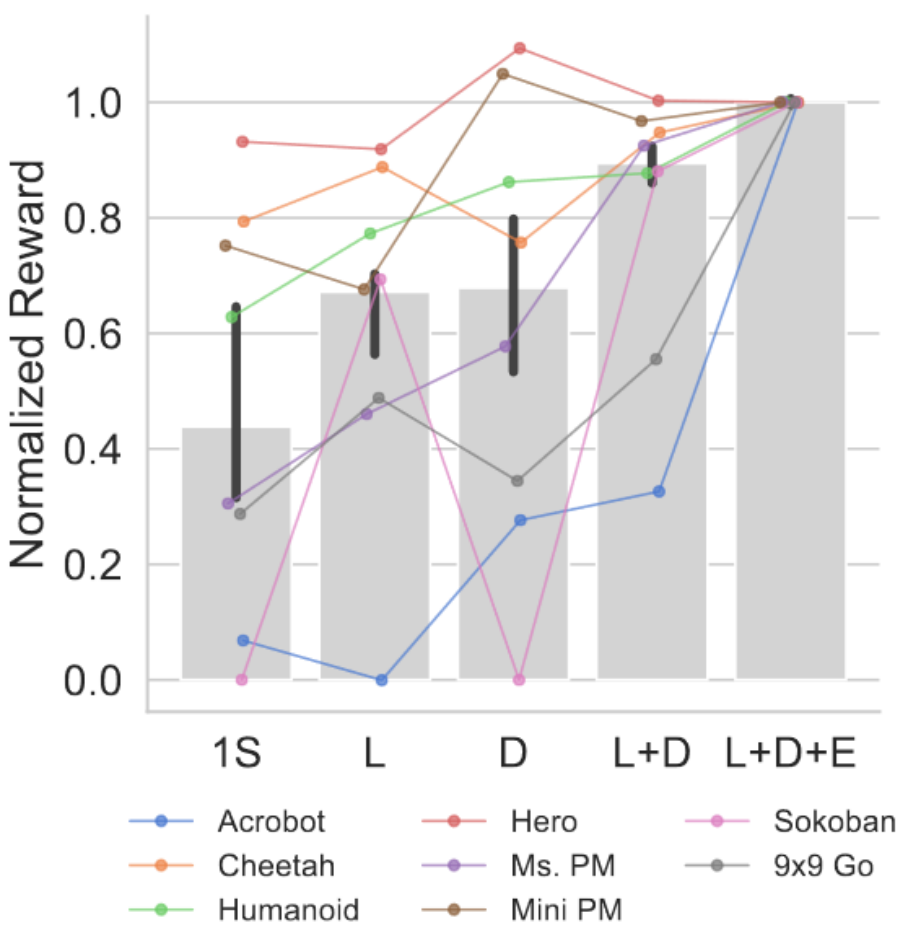
# Experimental Setting



- **One-Step**. This variant uses $D_{\text{tree}} = 1$ for learning, and acts by sampling from the policy prior. We also only train the model to predict one time step into the future (in all other variants, we train it to predict five time steps into the future). This variant is therefore as close as possible to a model-free version of MuZero, and could in principle be implemented solely with a Q-function.
- **Learn**. This variant uses $D_{\text{tree}} = \infty$ for learning, and acts by sampling from the policy prior. It therefore gets the benefits of a deep tree search for learning, but not for acting.
- **Data**. This variant uses $D_{\text{tree}} = 1$ for learning, acts during training by sampling from $\pi^{\text{MCTS}}$ (computed using $D_{\text{tree}} = \infty$), and acts at test time from the policy prior. This allows us to measure the impact of deep search on the distribution of data experienced during learning.
- **Learn+Data**. This variant using $D_{\text{tree}} = \infty$ both for learning and for acting during training; actions at test time are sampled from the policy prior. This variant thus benefits from search in two ways during learning, but uses no search at test time.
- **Learn+Data+Eval**. This variant is equivalent to the original version of MuZero, using search (with $D_{\text{tree}} = \infty$) for learning and for all action selection.

| | Train Update | Train Act | Test Act |
|---|---|---|---|
| **1S** | $D_{\text{tree}} = 1$ | $\pi_{\theta,t}$ | $\pi_{\theta,t}$ |
| **L** | $D_{\text{tree}} = \infty$ | $\pi_{\theta,t}$ | $\pi_{\theta,t}$ |
| **D** | $D_{\text{tree}} = 1$ | $D_{\text{tree}} = \infty$ | $\pi_{\theta,t}$ |
| **L+D** | $D_{\text{tree}} = \infty$ | $D_{\text{tree}} = \infty$ | $\pi_{\theta,t}$ |
| **L+D+E** | $D_{\text{tree}} = \infty$ | $D_{\text{tree}} = \infty$ | $D_{\text{tree}} = \infty$ |

# Overall Contributions




Search budget of B = 10 simulations in Minipacman, B = 25 in Sokoban, B = 150 in 9x9 Go, and B = 50 in all other environments.

- An important benefit of search is for policy learning
- Search at evaluation time only provides a small boost in most environments, though it occasionally proves to be crucial, as in Acrobot and Go

| | Train Update | Train Act | Test Act |
|---|---|---|---|
| **1S** | $D_{\text{tree}} = 1$ | $\pi_{\theta,t}$ | $\pi_{\theta,t}$ |
| **L** | $D_{\text{tree}} = \infty$ | $\pi_{\theta,t}$ | $\pi_{\theta,t}$ |
| **D** | $D_{\text{tree}} = 1$ | $D_{\text{tree}} = \infty$ | $\pi_{\theta,t}$ |
| **L+D** | $D_{\text{tree}} = \infty$ | $D_{\text{tree}} = \infty$ | $\pi_{\theta,t}$ |
| **L+D+E** | $D_{\text{tree}} = \infty$ | $D_{\text{tree}} = \infty$ | $D_{\text{tree}} = \infty$ |

Figure 3: Contributions of planning to performance, where 0.0 is the performance attained by a randomly initialized policy (Table 8), and 1.0 that obtained by full MuZero (Table 7). Grey bars show medians across environments, and error bars show 95% confidence intervals of the median. 1S=One-Step, L=Learn, D=Data, E=Eval. See Figure 10 for environment error bars.

# Planning for learning

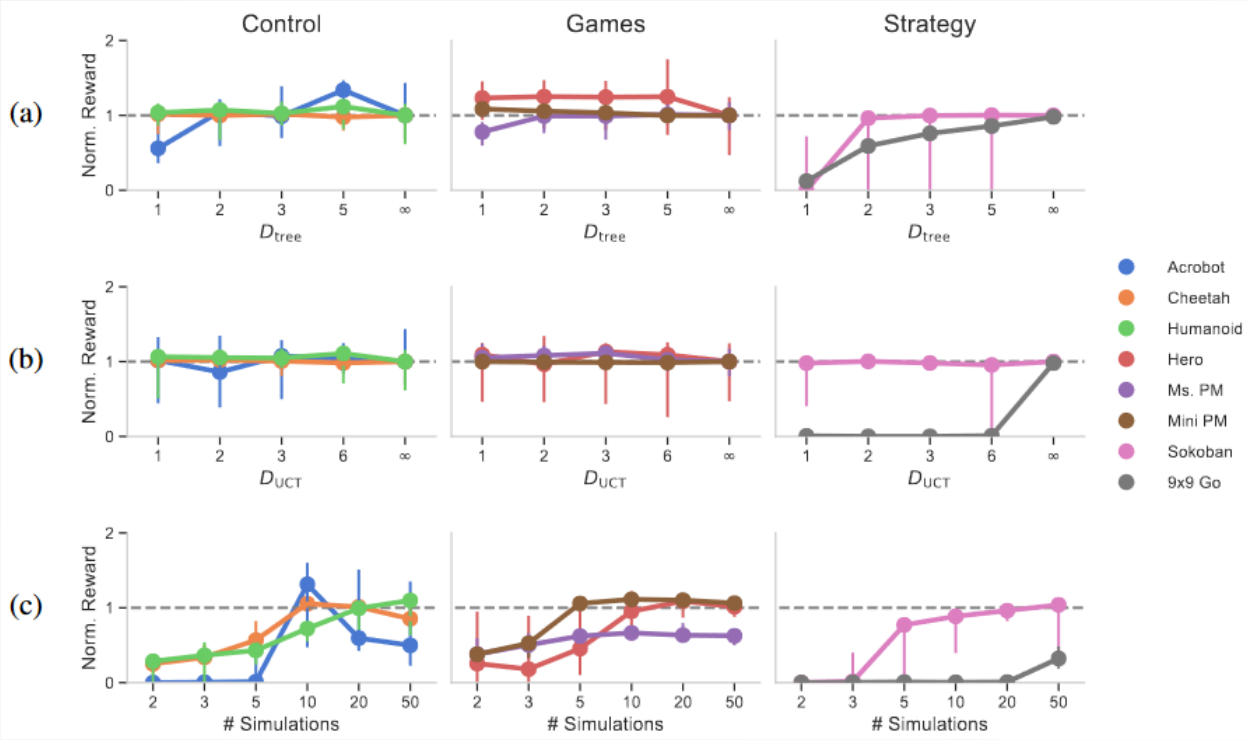



Figure 4: Effect of design choices on the strength of the policy prior. All colored lines show median normalized reward across ten seeds (except Go, which uses five seeds), with error bars indicating min and max seeds. Rewards are normalized by the median scores in Table 9. All agents use search for learning and acting during training only. (a) Reward as a function of $D_{\text{tree}}$. Here, $D_{\text{UCT}} = D_{\text{tree}}$ and the number of simulations is the same as the baseline. (b) Reward as a function of $D_{\text{UCT}}$. Here, $D_{\text{tree}} = \infty$ and the number of simulations is the same as the baseline. (c) Reward as a function of search budget during learning. Here, $D_{\text{UCT}} = 1$ and $D_{\text{tree}} = \infty$ (except in Go, where $D_{\text{UCT}} = \infty$).

- $D_{tree}$ does not make much of a difference in most environments, suggesting that deep tree search may not be necessary for learning a strong policy prior.
- $D_{UCT}$ has no effect except in 9x9 Go. Thus exploration-exploitation deep within the search tree does not seem to matter except in the most challenging settings.
- Some amount of planning can be beneficial, but too much can harm performance.

# Generalization in planning

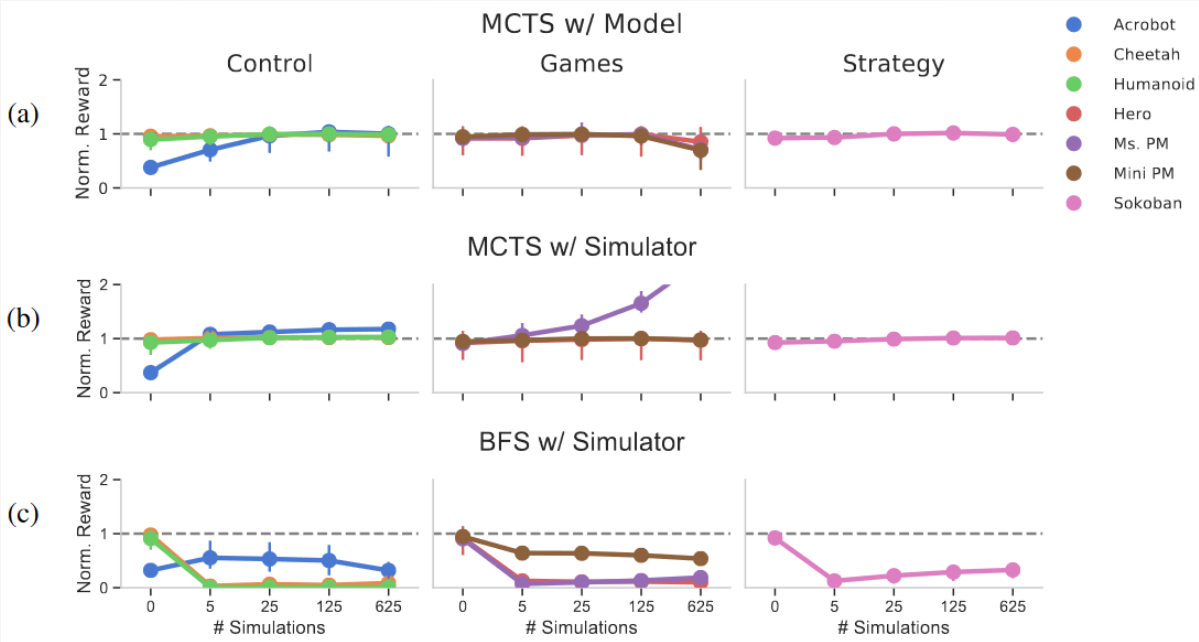



Figure 5: Effect of search at evaluation as a function of the number of simulations, normalized by the median scores in Table 10. All colored lines show medians across seeds, with error bars indicating min and max seeds. (a) MCTS with the learned model. (b) MCTS with the environment simulator. (c) Breadth-first search (BFS) with the environment simulator. Results with the learned model are similar and can be seen in Figure 11.

- Model generalization to new search budgets: A small improvement in performance between full MuZero without search. The reward at 625 simulations is also less than the baseline, indicating an effect of compounding model errors.
- Policy and value generalization with a better model: Planning with the simulator yields somewhat better results than planning with the learned model.
- Model generalization to new planners: BFS exhibiting a catastrophic drop in performance with any amount of search, suggests a mismatch between the value function and the policy prior.

# Generalizing to new mazes

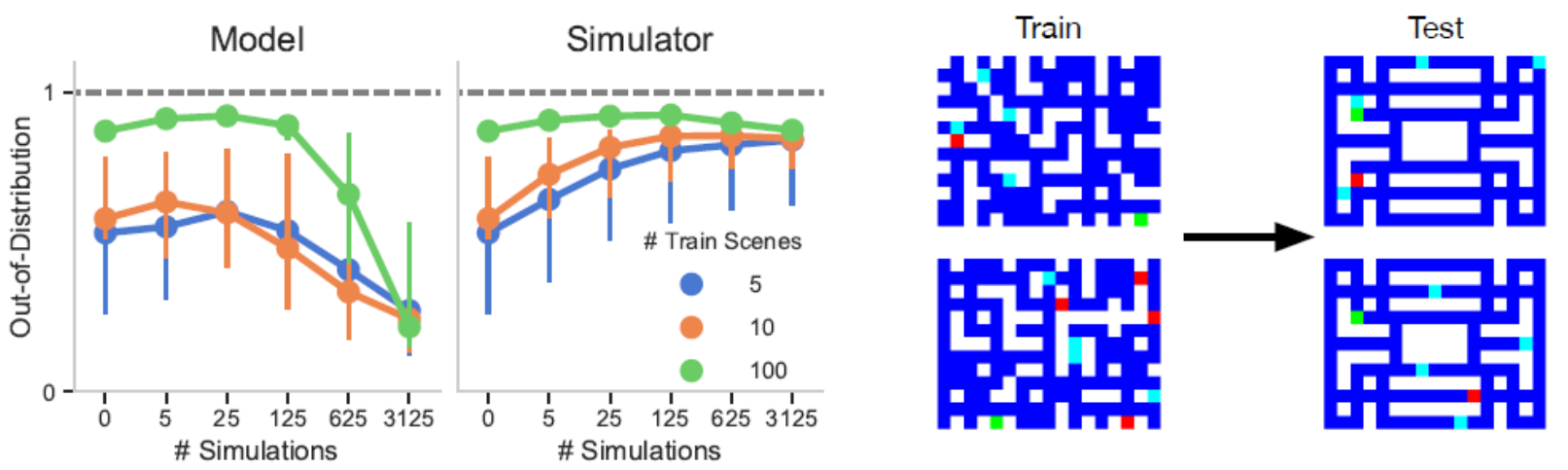



Figure 6: Generalization to out-of-distribution mazes in Minipacman. All points are medians across seeds (normalized by the median scores in Table 10), with error bars showing min and max seeds. Colors indicate agents trained on different numbers of unique mazes. The dotted lines indicate the baseline. The maps on the right give examples of the types of mazes seen during train and test. In-distribution generalization is shown in Figure 12 (Appendix) as the behavior is similar.

- A sharp drop-off in performance after that reflecting compounding model errors in longer trajectories.
- Reward using the simulator begins to decrease at 3125 simulations, indicating a sensitivity to errors in the value function and policy prior.

# Conclusion

- Search is most useful in constructing targets for policy learning and for generating a more informative data distribution.
- Simpler and shallower planning is often as performant as more complex planning. It suggests that many environments used in MBRL may not be fully testing the ability of model-based agents to perform sophisticated reasoning.
- Search at evaluation time only slightly improves zero-shot generalization, and even then only if the model is highly accurate.
- Search in challenging environments is ineffective without a strong value function or policy to guide it. If the value function and policy themselves do not generalize, then generalization to new settings will also suffer. But, if the value function and policy do generalize, then it is unclear whether a model is even needed for generalization.

中国科学院
自动化研究所
INSTITUTE OF AUTOMATION
CHINESE ACADEMY OF SCIENCES

# THANKS